

## Práctico 6 - Modularización; Alcance de Nombres y Archivos de texto

### Archivos de Texto

Si tiene problemas para trabajar con archivos, porque Python le tira un error de que no los encuentra, agregue a su programa el siguiente código (*en la parte de arriba, mantenga su código ordenado*):

---

```
import sys

sys.path.append(r".")
```

---

1. Mostrar el contenido en pantalla del archivo de texto llamado `fragmento_hobbit.txt`. (*Utilizar el archivo de texto que figura en el aula*) Determinar cuántas líneas de texto hay, mediante código, y mostrar en pantalla.
2. Crear un archivo de texto con el bloc de notas de Windows, guardarlo en la misma carpeta donde tiene su programa. Guardarle como contenido, una palabra por línea, inventadas, unas 10 líneas. Crear luego un programa que muestre en pantalla, una por una, las palabras que hay guardadas en ese archivo. Mostrar además, el total de líneas que había en el archivo.
3. Crear un programa que genere un archivo de texto llamado `numeros.txt` con 10 números enteros guardados en el mismo, uno por línea.
4. Crear un programa que le pida al usuario 5 colores, y los guarde en un archivo de texto llamado `colores.txt`
5. \* Crear un programa que dado un archivo de números con valores entre 1 y 10 (*lo puede generar como ud. desee*) determine cuántos números iguales a 5 hay en el archivo.
6. Crear un programa que dado un archivo de texto, que contiene números (*lo puede generar como ud. desee*) determine el valor promedio y la suma de todos ellos.
7. \* Crear un programita que permita elegir un color de una lista de 10 colores. Por defecto, el programa la primera vez que se inicia, empieza mostrando el color en pantalla azul (*como texto, 'azul'*), luego, cada vez que el programa sea ejecutado mostrará en pantalla el color que el usuario había seleccionado en la ejecución anterior del programa.
8. \* Generar dos archivos diferentes, uno llamado `pesos.txt` que contendrá 50 valores para designar los pesos de 50 personas. Otro llamado `alturas.txt` que contendrá para dichas 50 personas, las alturas en cm correspondientes. Generar un tercer archivo, llamado `bmi.txt` que tendrá calculados los BMI de cada persona. Permitir al usuario ingresar un número  $n$  (*del 1 al 50*) para mostrarle el bmi, traído desde el archivo de `bmi.txt` que ya generó usted, de la persona número  $n$ .

## Procedimientos

9. \* Crear un procedimiento que se encargue de imprimir un menu de opciones en pantalla. Puede elegir entre un menu de bebidas de un kiosco, un menu de pizzas, o un menu de operaciones aritmeticas a realizar entre dos números. El menu debe tener al menos unas 5 opciones, e incluir además, una opción de salir.
10. \* Realizar un programa que permita al usuario ingresar una longitud e imprima en pantalla un rectángulo de numerales, hueco por dentro. Para ello, deberá implementar un procedimiento llamado *rectangulo* que tome como parámetro un valor numérico, llamado longitud. Tendrá la misma cantidad de filas que de columnas, pero por cuestiones del espacio definido para las filas y los caracteres, se verá como un rectángulo y no como un cuadrado.  
Por ejemplo, si se ingresó 4 para la longitud, se verá en pantalla: *Tip: Puede ser útil pensarlo por línea horizontal*

---

```
####  
#  #  
#  #  
####
```

---

11. \* Escribir un procedimiento que tome dos números, inicio y fin, e imprima en pantalla la secuencia desde inicio hasta fin en la izquierda, y a su lado la secuencia inversa fin hasta inicio. Ejemplo, supóngase se le da inicio = 1 y fin = 99, entonces se verá en pantalla:

---

```
1 ——— 99  
2 ——— 98  
3 ——— 97  
.  
.  
.  
.  
.  
.  
.  
98 ——— 2  
99 ——— 1
```

---

## Alcance de Variables e Identificadores

En esta sección, romperemos reglas de buena escritura de código, porque no estamos escribiendo programas sino intentando comprender el ámbito de vida de los identificadores.

12. \* Determine si este fragmento de código da error, y en caso afirmativo, explicar porqué.

---

```
def imprimir_nro():
    n = 1
    print("El primer numero es: ", n)

imprimir_nro()
print("El primer numero es: ", n)
```

---

13. \* ¿Qué imprimirá en pantalla el siguiente código? ¿Cuál es el alcance de la variable frase? Además, si tuviese que evaluar la correctitud de este segmento de código, ud. diría que ¿es correcto?

---

```
frase = "Hola"

def proc():
    frase = "Es un lindo dia"

print(frase)
```

---

14. Determine cuáles variables son locales y cuáles son globales en el siguiente fragmento de código.

---

```
saludo = "Hola"

def saludar_mundo():
    mundo = "Mundo!"
    print(saludo, mundo)

def saludar_nombre(nombre):
    print(saludo, nombre)

saludar_mundo()
saludar_nombre("Totoro")
```

---

15. Dé un ejemplo pequeño de código, con una función o procedimiento inventado por usted, que tenga variables locales y globales.

16. ¿Qué imprimirá en pantalla el siguiente código? Determine el alcance de cada variable.

```
x = 3

def p1():
    y = x + 1
    print(x)

    def p2():
        x = 1
        print(y)
        print(x)

    p2()

p1()
```

## Integrando conceptos

17. \* Generar un archivo de texto, llamado datos.txt, que constará de las siguientes columnas, cada una, separada por coma: x, y  
A modo ejemplo, así se verían los valores dentro del archivo: *(observar que los puntos son el separador decimal de los números, y que la coma representa una coma del lenguaje que separa un número de otro, no refiere a coma decimal. Recuerde que los números con decimales en la programación son con punto, no con coma.)*

```
0.2,0.34
0.5,0.8
.
.
.
```

Los valores de  $x$  iniciarán en 0.2 hasta llegar a 100.5, en incrementos de 0.3 Los valores de  $y$  serán generados por la función sigmoidea

$$y(x) = \frac{1}{1 + e^{-x}}$$

Una vez generado el archivo, su programa deberá leer sus datos y graficarlos.